

CS 252:

Advanced Programming Language Principles



Higher-Order Functions Part 2: Fold

Prof. Tom Austin

San José State University

Review: map, zipWith, and filter

```
map :: (a -> b) -> [a] -> [b]
```

```
zipWith :: (a -> b -> c) ->
           [a] -> [b] -> [c]
```

```
filter :: (a -> Bool) -> [a] -> [a]
```

map

```
Prelude> map (+1) [1,2,3]
[2,3,4]
Prelude> map show [1,2,3]
["1","2","3"]
Prelude> let squares = map (\x -> x*x) [1..]
Prelude> take 10 squares
[1,4,9,16,25,36,49,64,81,100]
Prelude>
```

zipWith

```
Prelude> map (+) [1,2,3] [3,2,1]
```

```
<interactive>:8:1: error:
```

- Couldn't match expected type '[a1] -> t'
with actual type '[a0 -> a0]'
- The function 'map' is applied to three arguments,
but its type '(a0 -> a0 -> a0) -> [a0] -> [a0 -> a0]'
has only two
In the expression: map (+) [1, 2, 3] [3, 2, 1]
In an equation for 'it': it = map (+) [1, 2, 3]
[3, 2, 1]
- Relevant bindings include it :: t
(bound at <interactive>:8:1)

```
Prelude> zipWith (+) [1,2,3] [3,2,1]
```

```
[4,4,4]
```

```
Prelude>
```

filter

```
Prelude> filter (>0) [-5..5]
```

```
[1,2,3,4,5]
```

```
Prelude> filter even $
```

```
    map read ["1","2","3","4","5"]
```

```
[2,4]
```

```
Prelude>
```

Review map/zipWith/filter lab.

Motivation for other
higher order functions.
(in-class)

Higher order functions

```
map :: (a -> b) -> [a] -> [b]
```

```
zipWith :: (a -> b -> c) ->
           [a] -> [b] -> [c]
```

```
filter :: (a -> Bool) -> [a] -> [a]
```

```
foldl :: (a -> b -> a) -> a -> [b] -> a
```

```
foldr :: (a -> b -> b) -> b -> [a] -> b
```

Fold left

`foldl` applies a function to each sequential pair of elements in a list

This is the
accumulator

- `foldl (\x y -> x+y) 0 [1, 2, 3]`
- `foldl (\x y -> x+y) (0+1) [2, 3]`
- `foldl (\x y -> x+y) ((0+1)+2) [3]`
- `foldl (\x y -> x+y) (((0+1)+2)+3) []`
- `((0+1)+2)+3)`
- 6

Fold right

`foldr` folds from the right, and works on infinite lists

Note that we can pass '+' as a function

- `foldr (+) 0 [1, 2, 3]`
- `1 + (foldr (+) 0 [2, 3])`
- `1 + (2 + (foldr (+) 0 [3]))`
- `1 + (2 + (3 + (foldr (+) 0 [])))`
- `1 + (2 + (3 + (0)))`
- `6`

foldr on an infinite list

- `take 3 $ foldr (:) [] [1..]`
- `take 3 $ 1:foldr (:) [] [2..]`
- `take 3 $ 1:2:foldr (:) [] [3..]`
- `take 3 $ 1:2:3:foldr (:) [] [4..]`
- `[1,2,3]`

`foldl` (& `foldr`) build a *thunk* rather than calculate the results as it goes.

```
> let z = foldl (+) 0 [1..10000000]
```

Returns quickly

```
> z
```

Slow – result needs to
be computed

Definition: a *thunk* is a
delayed computation.

`foldl'` – Efficient left fold

- `foldl'` evaluates its results *eagerly* rather than *lazily*.
- To use, first:

```
import Data.List
```
- https://wiki.haskell.org/Foldr_Foldl_Foldl' has more details.

Lazy fold left steps

- `foldl (\x y -> x+y) 0 [1,2,3]`
- `foldl (\x y -> x+y) (0+1) [2,3]`
- `foldl (\x y -> x+y) ((0+1)+2) [3]`
- `foldl (\x y -> x+y) (((0+1)+2)+3) []`
- `((0+1)+2)+3)`
- 6

Eager fold left steps

- `foldl' (\x y -> x+y) 0 [1,2,3]`
- `foldl' (\x y -> x+y) (0+1) [2,3]`
- `foldl' (\x y -> x+y) 1 [2,3]`
- `foldl' (\x y -> x+y) (1+2) [3]`
- `foldl' (\x y -> x+y) 3 [3]`
- `foldl' (\x y -> x+y) (3+3) []`
- `(3+3)`
- `6`

Which fold should I use?

- **foldr** – "foldr is not only the right fold, it is almost commonly the *right* fold to use..."
 - (Not the case in non-lazy languages)
- **foldl'** – large, but finite lists
- **foldl** – specialized cases only

Related reading

- Learn You a Haskell, Chapter 6 (online)
- https://wiki.haskell.org/Foldr_Foldl_Foldl'
- https://wiki.haskell.org/Foldl_as_foldr

Lab 4: Higher order functions

Available in Canvas and on the course website.